Security Assessment — Stratroom-Source

Date: 2026-06-13
Scope: `stack-service` (authservice, db-service, stratroom-backend) — Spring Boot microservices
Assessment type: Static source-code review (defensive)

Executive Summary

The application's security is undermined by a single recurring pattern: **secrets are hardcoded in plaintext with weak, guessable default values**, repeated across JWT signing, database access, Jasypt encryption, SMTP, and LDAP. The most severe consequence is that the JWT signing secret (`123456`) allows **any attacker to forge valid authentication tokens for any user**, defeating the entire authentication system.

A second major risk is the **"Lite" controller**, which grants every caller full Super-Admin permissions with no authentication. If deployed outside local demos, it is a complete authorization bypass.

Severity	Count
🔴 Critical	3
🟠 High	3
🟡 Medium	3

🔴 Critical Findings

C-1. Hardcoded, trivially-guessable JWT signing secret → full authentication bypass

Location: `authservice/.../config/EmbeddedTomcatConfig.java:44`, `stratroom-backend/.../application.properties:46`

```
jwt.secret=123456
```

Tokens are signed with `HMAC256(secret)` (`JwtTokenUtil.java:29`). With a secret of `123456`, an attacker can forge a valid JWT for any user and access every protected endpoint.

Impact: Complete authentication bypass; full account/data compromise.
Fix: Use a long (≥256-bit) random secret injected via environment variable or secrets manager. Remove the weak default. Rotate immediately.

C-2. "Lite" controller grants full Super-Admin access to everyone

Location: `db-service/.../DbServiceLiteController.java:23-99`

- `validateLicense` → always returns `validationSuccess=true`
- `getUserRole` → always returns `"Super Admin"`
- `getUserPermissions` → returns View/Create/Update/Delete on **every module** for any `empId`, with no authentication.

Impact: If this lite build reaches a real environment (just a different main class), it is a total authorization bypass.
Fix: Guard the controller behind a Spring profile (`@Profile("lite")`) so it cannot load in production; never deploy lite builds externally.

C-3. Database credentials in plaintext, using `root`

Location: `db-service/db-service.properties:8-9`, `stratroom-backend/.../application.properties:15-16`

```
spring.datasource.username=root
spring.datasource.password=123456   # and "1234"
```

Impact: DB superuser with a weak password committed to source. Any config/repo leak exposes the entire database.
Fix: Use a non-`root`, least-privilege DB user with a strong password. Move credentials to environment variables / secrets manager.

🟠 High Findings

H-1. Weak Jasypt encryption with guessable password

Location: `authservice/.../config/EmbeddedTomcatConfig.java:26,35`

```
auth.jasypt.algorithm=PBEWithMD5AndDes   # MD5 + single DES — cryptographically broken
auth.jasypt.password=123456
```

Impact: Anything encrypted (e.g. the `userInfo` payload) can be decrypted or forged.
Fix: Use `PBEWITHHMACSHA512ANDAES_256` (or stronger) with a strong, externalized password.

H-2. Unencrypted database traffic + MITM password leak

Location: `stratroom-backend/.../application.properties:13`

```
useSSL=false&allowPublicKeyRetrieval=true
```

Impact: DB traffic is cleartext; `allowPublicKeyRetrieval=true` lets a man-in-the-middle capture the MySQL password.
Fix: Set `useSSL=true` with proper certificate verification; remove `allowPublicKeyRetrieval=true`.

H-3. Additional plaintext secrets committed

Location: `stratroom-backend/.../application.properties:59` (SMTP), `:38` (LDAP bind password)

The LDAP bind password is consumed directly in `db-service/.../config/LdapConfig.java:36`.

Impact: SMTP and LDAP credentials exposed in source control.
Fix: Externalize all secrets; rotate any that have been committed.

🟡 Medium Findings

M-1. Exception details leaked to caller

Location: `authservice/.../oauth/CustomAuthenticationProvider.java:94`

```
throw new RuntimeException("Authentication failed: " + exp.getMessage());
```

Impact: Internal error/stack details surfaced to clients (information disclosure).
Fix: Log internally; return a generic authentication-failure message.

M-2. No token revocation; long refresh window

Location: `authservice/.../jwt/JwtTokenUtil.java:20`

Refresh tokens last 7200s with no denylist/revocation. A stolen or forged token is valid until expiry.
Fix: Add token revocation (denylist or short-lived access tokens + rotating refresh tokens).

M-3. Machine-specific absolute path in config

Location: `db-service/db-service.properties:14`

```
license.file.path=C:/Users/sibi/Desktop/Stratroom-Source/license.txt
```

Impact: Leaks a developer username; breaks portability (config smell).
Fix: Use a relative or environment-configured path.

Recommended Remediation Order

1. **Replace** `jwt.secret` with a long random value via environment variable — remove the `123456` default. *(C-1)*
2. **Gate the Lite controller** behind `@Profile("lite")`; never deploy lite builds externally. *(C-2)*
3. **Externalize all credentials** (DB, JWT, Jasypt, SMTP, LDAP) to env vars / secrets manager; use a non-`root` DB user. *(C-3, H-3)*
4. **Enable TLS** for DB connections and **upgrade Jasypt** to an AES-256 algorithm. *(H-1, H-2)*
5. **Stop echoing exception messages** to clients. *(M-1)*

Root cause

The dominant structural problem is **secrets committed in plaintext with weak default values**, repeated across every subsystem. Fixing this one habit — externalize and rotate all secrets — closes most of the critical attack surface.

This document reflects a static review of the source as of the assessment date. It does not include dynamic/runtime testing.